

基于单片机设计实现——应用系统

设计方案

2026 年 10 月 7 日

目录

1	单片机 ESP32-S3 简介	1
1.1	硬件设计原理图	1
1.2	MCU 介绍	2
1.2.1	系统	2
1.2.2	外设	3
1.3	OV2640 摄像头	4
2	设计思路	4
2.1	方案雏形	5
2.2	初版方案	5
2.3	终版方案	6
3	设计功能点	7
3.1	方案雏形	7
3.2	初版方案	7
3.3	终版方案	8
4	软件设计	8
4.1	方案雏形	8
4.1.1	初版方案	11
4.2	终版方案	18
5	运行效果	23
5.1	初版方案	24
5.2	终版方案	24

1 单片机 ESP32-S3 简介

1.1 硬件设计原理图

以下是 MCU 硬件设计原理图。本图在乐鑫官方文档原理图 [2] 的基础上，做出了如下修改并重新绘制：

涉及保留的部分：

- 保留 ESP32-S3 芯片主控 (U1) 即芯片主体及基本的电源脚。
- 保留 VDD33 电源滤波和去耦电容。
- 保留主时钟晶振电路。
- 保留存储固件程序 (U2) 与 PSRAM (U3)。
- 保留复位电路 (CHIP_PU)。

涉及舍弃的部分：

- 射频天线匹配网络等涉及 Wi-Fi 和蓝牙的部分。
- JTAG 调试引脚等可以被 USB 代替烧录调试的部分。

涉及增添的部分：

- USB Type-C 接口和供电电路。
- 24-Pin FPC 摄像头 (OV2640) 插座：添加一个 24 针脚的 FPC 翻盖座子。此处依据软件完成引脚映射与芯片连接。

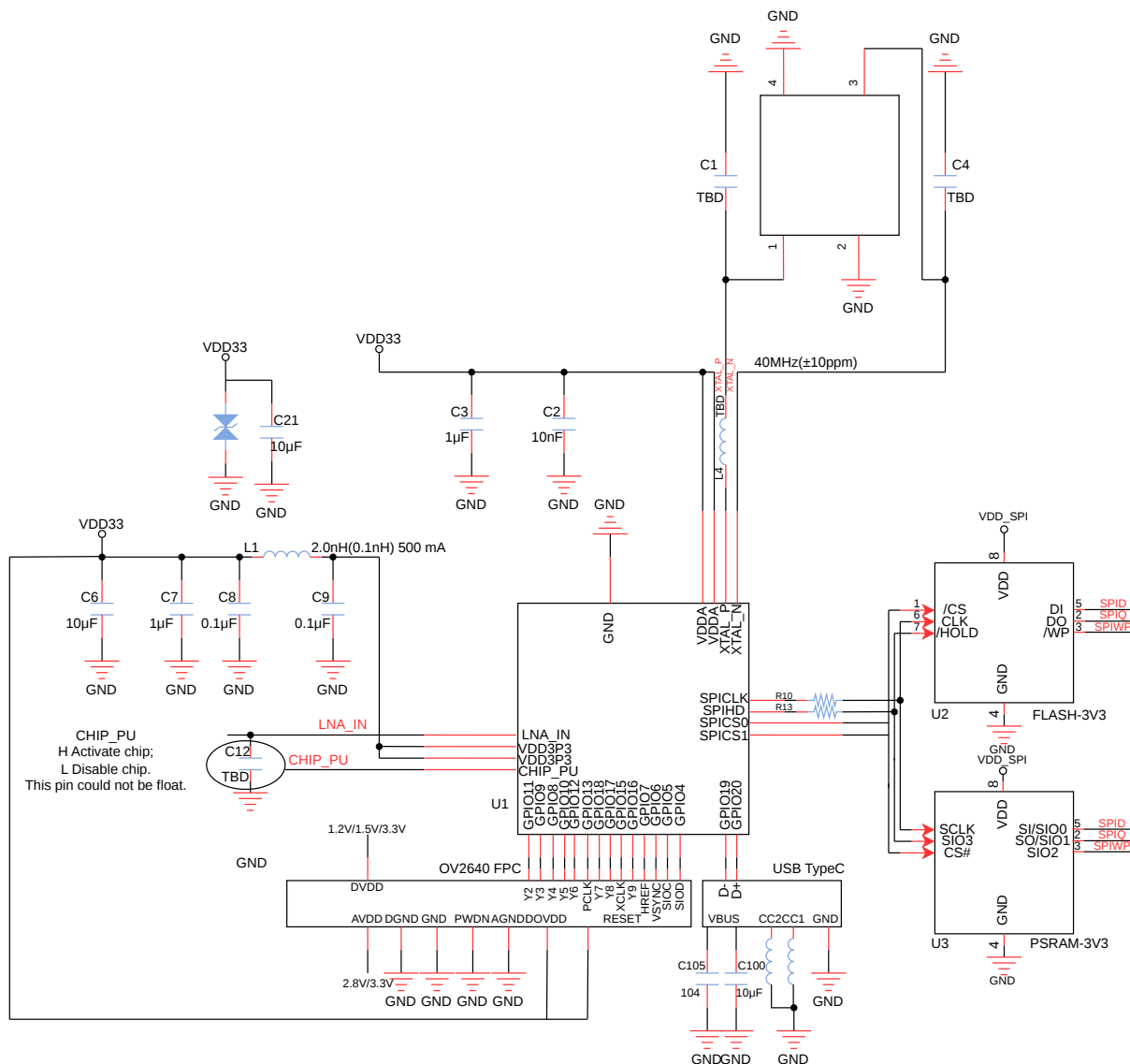


图 1.1: 原理图

1.2 MCU 介绍

ESP32-S3 本质上是一颗高集成度的 SoC，因为其系统目标就是处理器，所以也可以视为一颗 MCU[3]。我们只介绍该 MCU 中本项目涉及到的功能。

1.2.1 系统

CPU

ESP32-S3 采用 Xtensa® LX7 32 位双核处理器，具有如下特性：

- 典型五级流水线架构，时钟频率达 240MHz。
- 采用 Xtensa 指令集，支持半/全定制化指令集扩展。

- 支持 32 位乘法器、32 位除法器，适合分布式计算。
- 支持六级 32 个中断。

指令扩展

针对 AI 边缘和数字信号处理算法的运算效率，ESP32-S3 新增了扩展指令和处理器功能。在原有 64 个物理通用寄存器的基础上新增 128 位宽通用寄存器，并提供 128 位宽的向量数据操作，合并数据处理指令与加载/存储运算指令。针对 128 位宽向量数据的潜在问题，底层实现了非对齐存取和取饱和操作。

内部存储器

ESP32-S3 的内部存储器即集成于芯片晶圆上或封装内部的存储器，包括 ROM、SRAM、eFuse 和 flash，各个存储元件具有不同特性：

1. 384KB ROM：用于程序启动和内核功能调用。
2. 512KB 片上 SRAM：用于数据和指令存储。
3. 8KB SRAM RTC 快速存储器：为处理器直接访问。
4. 8KB SRAM RTC 慢速存储器：为处理器直接访问。
5. 4096 位 eFuse 存储器。
6. Flash、PSRAM：通过高频 SPI 总线外挂，后者是本项目用于图像缓冲区的重要组成部分。

1.2.2 外设

UART 控制器

ESP32-S3 有三个 UART 控制器，支持异步通信和 IrDA，通信速率可达到 5Mbps。

I2C 接口

ESP32-S3 有两个 I2C 总线接口，可以用作 I2C 主机或从机模式。

LCD 与 Camera 控制器

LCD 模块用于发送并行视频数据信号，其总线 8 位 16 位并行 RGB、I8080、MOTO6800 接口，支持 RGB565、YUV422、YUV420、YUV411 之间的互相转换。

Camera 模块用于接收并行视频数据信号，其总线支持 8 位 16 位 DVP 图像传感器接口，支持 RGB565、YUV422、YUV420、YUV411 之间的互相转换。

在本项目中，OV2640 每一个像素时钟 (PCLK) 在 D0 D7 上并行输出一个字节的数
据，ESP32-S3 内部的 Camera 模块通过 DVP 控制器以硬件直接存取 (DMA) 的方式，将这些像素数据高速搬运到 PSRAM 内存中。此外 OV2640 输出的原始格式通常是 RGB565。而本项目采用的 MNIST 神经网络模型需要的是 28×28 单通道灰度图。Camera 模块可以在像素进入内存时直接进行色彩空间转换，从而降低 CPU 逐像素计算的软件开销。

USB 2.0 OTG 全速接口

ESP32-S3 带有一个集成了收发器的全速 USBOTG 外设，符合 USB2.0 规范。

我们会在讲解 Type-C 接口的时候再详细叙述。

1.3 OV2640 摄像头

本外设是在购买 ESP32-S3 开发板时附带的摄像头模块，其出厂即通过 FPC 柔性电路软板被翻盖锁扣锁定，撕掉镜头底座背面的双面胶纸后，即可将镜头按压粘在开发板的 PCB 板面上。此时还需要撕掉镜头镜头盖正面的微型透明防尘保护膜。

参数

OV2640 采用 1/4 英寸 CMOS 图像传感器，最高支持 200 万像素、每秒 15 60 帧图像采集。在本项目中，我们将其降采样为 28×28 以适应 MNIST 软件特征。借助片上 ISP，OV2640 可以通过自身 ISP Engine 在硬件层直接完成自动曝光、自动增益、自动白平衡和 JPEG 格式压缩，极大地减轻了 ESP32-S3 的计算负担。

接口

如原理图所示：并行数据线 $Y2 \sim Y9$ 即 DVP 8 位数据总线 ($D0 \sim D7$)，连接 ESP32-S3 的 GPIO11, 9, 8, 10, 12, 18, 17, 16； $PCLK$ (Pixel Clock) 连接像素时钟线 (GPIO13)。每当 $PCLK$ 时钟脉冲跳变一次， $D0 \sim D7$ 的 8 根引脚上就会同时输出 1 个字节的像素数据。这里的图像是 RGB565 格式，当 $PCLK$ 敲击两次， $D0 \sim D7$ 就会吐出 2 个字节，ESP32-S3 的 DMA 控制器就会自动把这 2 个字节组合成一个完整的像素点写入 PSRAM 内存。上述分析说明了本项目的采样频率和图像特点。

OV2640 上的 $VSYNC$ 连接 GPIO6， $HREF$ 连接 GPIO7。当摄像头准备传输新的一帧画面时， $VSYNC$ 将会拉低/拉高该引脚脉冲。当摄像头准备传输画面中的新一行像素时，将会激活 $HREF$ 。这个原理和计算机系统综合设计中的 VGA 显示器显示原理是类似的。ESP32-S3 的 Camera 控制器依靠这两个硬件信号，把一连串连续流动的字节流，精准还原为内存中结构化的二维图像矩阵。

主时钟 $XCLK$ 连接 GPIO15。OV2640 芯片内部没有晶振，其内部时钟须依赖 ESP32-S3 驱动一个 $10MHz \sim 24MHz$ 的方波信号。ESP32-S3 先输出 $XCLK$ 心跳激活 OV2640，OV2640 才能根据这个内部倍频，进而产生 $PCLK$ 和像素数据提供给主控。

2 设计思路

本项目旨在探索受限资源单片机在机器视觉与人工智能领域的应用。在硬件平台选择上，80C51 较难胜任算力要求和内存要求高的数字图像识别任务。

ESP32-S3 具备 AI 加速指令集且具有可选 PSRAM 处理图像缓冲问题。

在环境配置阶段，选用了 Arduino IDE，平台信息会在软件部分叙述。克服工具链路径依赖问题后，我们成功构建了基于 C++ 的底层硬件驱动环境，为后续的软硬件协同

开发奠定了基础。

2.1 方案雏形

Arduion IDE 提供了对 esp32 by espressif 下载源的支持。项目初期，通过部署 esp32 库提供的 CameraWebServer 示例，验证了 OV2640 摄像头的数据采集能力。单片机作为图像采集节点，可以将视频流通过 Wi-Fi 传输至局域网内的 PC 端。具体来说，如果示例占用了 X 端口，可以通过 `http://localhost:X/picture` 直接获取图像。尝试调用经典的 MNIST 神经网络模型进行阿拉伯数字的识别，我成功跑通了数字图像识别。

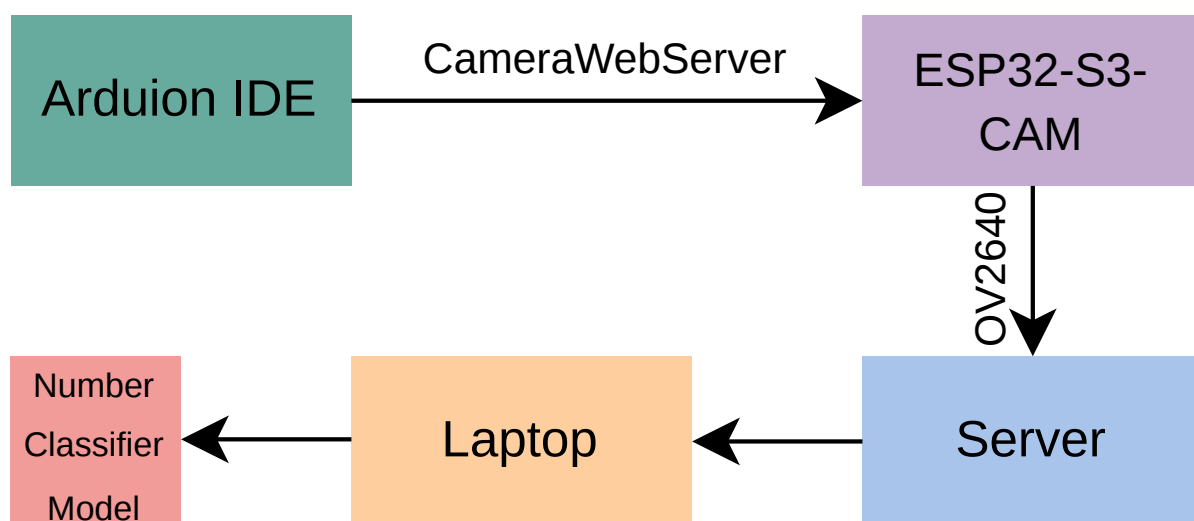


图 2.1: 方案雏形

这种传统的云/端分离架构存在缺陷：

- 高度依赖 Wi-Fi 网络稳定性。
- 存在明显的网络传输延迟。

2.2 初版方案

注意到 MNIST 模型参数量极低，因此 AI 推理过程可以从 PC 端下放至 MCU 侧，实现真正的边缘计算。

具体来说，摄像头输出灰度图像，直接存入 ESP32-S3 的 PSRAM 帧缓冲区。设计裁剪逻辑提取 ROI (Region of Interest)，从而下采样至 28x28 的特征张量。引入二值化反转算法，将现实中的白纸黑字动态转换为模型所需的黑底白字特征。最后将预处理后的张量送入通过 Edge Impulse 打包的片上 ONNX 视觉模型，提取 Logit 最高分映射为实际数字。本质上是利用 ESP32-S3 的双核 240MHz 算力与片上内存，直接在本地运行量化后的神经网络。

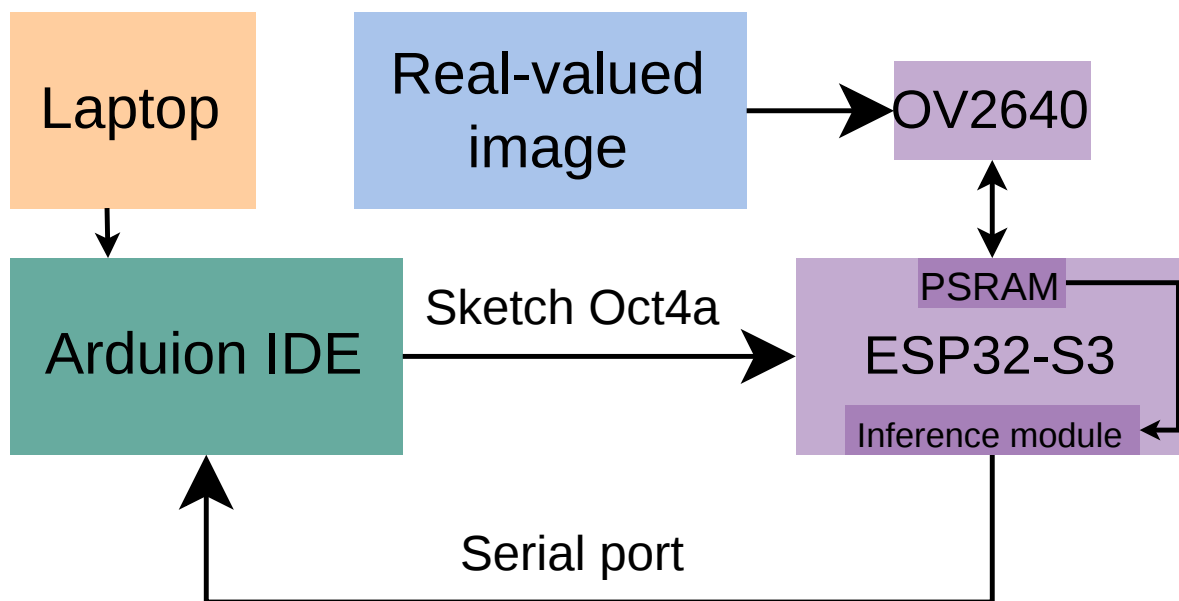


图 2.2: 初版方案

该方案成功实现了脱机的本地视觉识别，推理耗时控制在数百毫秒级，证明了 MCU 运行 AI 模型的完全可行性。作为初版方案，在实际测试中发现，传统的图像分类模型对数字在画面中的位置极其敏感，必须严格对准画面中心才能准确识别，鲁棒性仍有提升空间。

2.3 终版方案

为了解决初版方案影响图像截取的物理限制，终版方案将计算机视觉任务从**图像分类**升级为**目标检测**。即引入专为微控制器设计的 FOMO 目标检测架构，摒弃了锚框机制，转而预测画面中的目标中心点。以实现摄像头在完整的视野范围内，同时检测并框出图像中的人脸与佩戴口罩信息，并输出是否佩戴口罩。这使得整个系统可以进阶为具备工业分拣、智能计数潜力的完整边缘视觉设备。

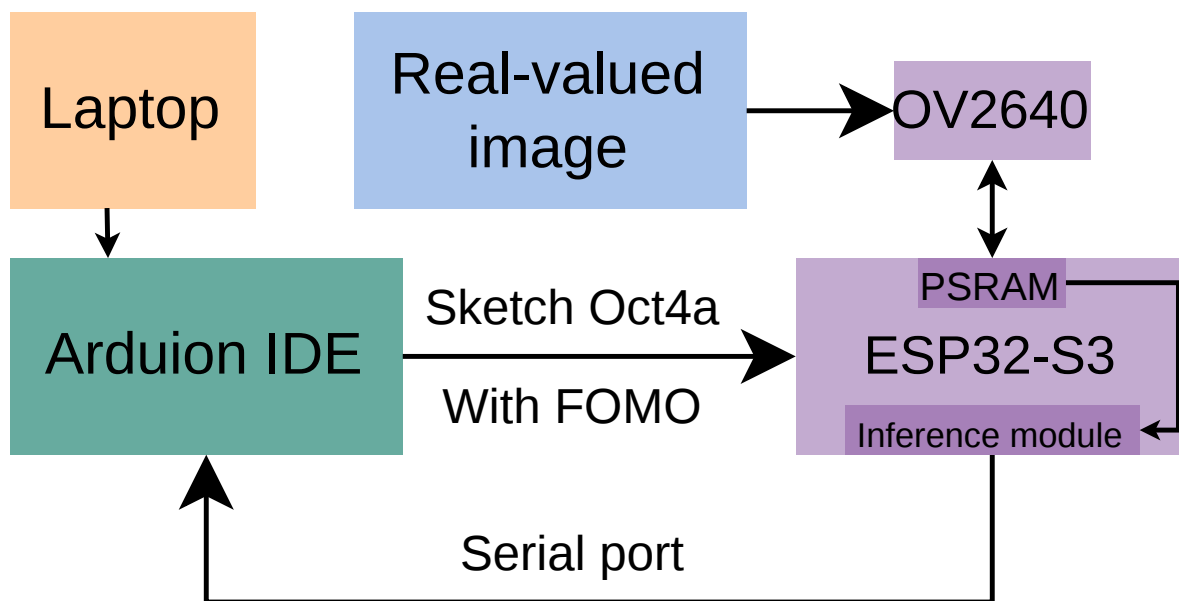


图 2.3: 终版方案

3 设计功能点

3.1 方案雏形

1. 图像实时推流：ESP32-S3-CAM 通过 Wi-Fi HTTP 协议将 OV2640 捕获的视频流推送到 PC 端。需要注意，OV2640 本身没有固定的端到端传输延迟：若帧率为 15 FPS 或 30 FPS，仅等待下一帧就分别约需 67 ms 或 33 ms；实际延迟还包括曝光与读出、JPEG 编码、ESP32 缓冲及 Wi-Fi/HTTP 传输和 PC 端显示。因而总延迟取决于分辨率、帧率、缓冲策略和网络状况。通常能维持每秒 15 帧。
2. 上位机辅助识别：PC 端接收视频流并调用 MNIST 模型进行阿拉伯数字分类与展示。由于不依赖单片机，这个过程特别快。

3.2 初版方案

1. 脱机本地推理：不依赖 Wi-Fi 传输，由 ESP32-S3 片上直接运行 MNIST 图像分类模型。
2. 硬件预处理：对采集到的图像进行中心 160x160 ROI 裁剪、28x28 缩放以及二值化黑白反转。
3. 串口数据反馈：实时通过 UART 串口输出识别出的数字类别、置信度 Score 和片上推理耗时。

上述初版方案识别一次结果通常耗时约 1400ms。

3.3 终版方案

1. 智能识别：取消中心裁剪限制，在完整视野范围内自动寻找人脸。
2. 多目标检测与定位：基于 FOMO 极轻量目标检测架构，支持识别并定位画面中人脸与是否佩戴口罩信息。
3. 串口数据反馈：串口输出推理结果与置信度外，实时吐出检测每个目标的耗时。

由于改进了模型导出策略，不同于初版方案全量识别，终版方案识别一次结果通常耗时约 140ms。

4 软件设计

本项目的软件侧涉及四个平台，如下呈现了它们各自的职责：

表 4.1: 软件侧平台职责

平台	职责
VisualStudioCode	模型获取、数据集获取、模型训练
Arduino IDE	确定硬件行为、导入位流
Edge Impulse	模型训练、模型打包
Roboflow Universe	获取数据集

后续会介绍各个方案的软件设计，会区分各个方案使用平台的不同。

4.1 方案雏形

首先需要说明该方案在 Arduion IDE 平台上的代码非本人所做，而是来自于 esp32 开发板环境示例 CameraWebServer。我在此基础上做了少量修改，只展示涉及修改的部分。

根据板子型号规定引脚使用

代码 4.1：定义引脚

```
1 /// board_config.h
2 #ifndef BOARD_CONFIG_H
3 #define BOARD_CONFIG_H
4
```

```

5 #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
6 #include "camera_pins.h"
7
8 #endif // BOARD_CONFIG_H
9
10 /// camera_pins.h
11 #if defined(CAMERA_MODEL_WROVER_KIT)
12 // ...
13
14 #elif defined(CAMERA_MODEL_ESP32S3_EYE)
15 #define PWDN_GPIO_NUM -1
16 #define RESET_GPIO_NUM -1
17 #define XCLK_GPIO_NUM 15
18 #define SIOD_GPIO_NUM 4
19 #define SIOC_GPIO_NUM 5
20
21 #define Y2_GPIO_NUM 11
22 #define Y3_GPIO_NUM 9
23 #define Y4_GPIO_NUM 8
24 #define Y5_GPIO_NUM 10
25 #define Y6_GPIO_NUM 12
26 #define Y7_GPIO_NUM 18
27 #define Y8_GPIO_NUM 17
28 #define Y9_GPIO_NUM 16
29
30 #define VSYNC_GPIO_NUM 6
31 #define HREF_GPIO_NUM 7
32 #define PCLK_GPIO_NUM 13

```

上述代码根据开发板原理图规定了摄像头的各个针脚与 ESP32-S3 的 GPIO 之间的映射关系。后续方案会以不同方式遵循这些针脚映射。

根据 CameraWebServer 的设计，硬件会返回 ESP32_IP，在 PC 端访问 http://ESP32_IP/picture 即可获得摄像头的 JPEG 图像。在本地实现 MNIST 模型推理的过程中，只需要将 JPEG 图像送入即可：

代码 4.2：本地模型推理

```

1 import cv2
2 import numpy as np
3 import urllib.request

```

```

4 from torchvision import transforms
5
6 onnx_url = "https://github.com/onnx/models/raw/main/validated/vision/
    classification/mnist/model/mnist-12.onnx"
7 model_path = "mnist-12.onnx"
8
9 # 下载 ONNX 模型
10 urllib.request.urlretrieve(onnx_url, model_path)
11
12 # 加载 ONNX 模型
13 net = cv2.dnn.readNetFromONNX(model_path)
14
15 # 读取图像并输入模型推理
16 def predict_digit(img_28x28):
17     # 归一化并转为 blob 格式 [1, 1, 28, 28]
18     blob = cv2.dnn.blobFromImage(img_28x28, 1.0 / 255.0, (28, 28), (0, 0, 0)
19         , swapRB=False, crop=False)
20     net.setInput(blob)
21     outputs = net.forward()
22     prediction = np.argmax(outputs)
23     confidence = np.max(cv2.softmax(outputs)) if hasattr(cv2, 'softmax')
24         else np.exp(outputs[0][prediction]) / np.sum(np.exp(outputs[0]))
25     return prediction, confidence
26
27 if __name__ == "__main__":
28     # 这是上次实验默认的 ESP32-S3-CAM IP 地址
29     ESP32_IP = "192.168.2.17"
30     URL = f"http://{ESP32_IP}/capture"
31
32     transform = transforms.Compose([
33         transforms.ToTensor(),
34         transforms.Normalize((0.1307,), (0.3081,))
35     ])
36
37     while True:
38         try:
39             # 抓取一帧 JPEG 图像
40             img_resp = urllib.request.urlopen(URL, timeout=5)
41             img_np = np.array(bytearray(img_resp.read()), dtype=np.uint8)
42             frame = cv2.imdecode(img_np, -1)

```

```

41
42     if frame is None:
43         continue
44
45     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46     blurred = cv2.GaussianBlur(gray, (5, 5), 0)
47     _, thresh = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY_INV
        + cv2.THRESH_OTSU)
48
49     resized = cv2.resize(thresh, (28, 28), interpolation=cv2.
        INTER_AREA)
50
51     prediction, confidence = predict_digit(resized)
52
53     display_frame = frame.copy()
54     text = f"Digit: {prediction} ({confidence*100:.1f}%)"
55     cv2.putText(display_frame, text, (20, 40), cv2.
        FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
56
57     cv2.imshow("ESP32-S3 Digital Recognition", display_frame)
58     cv2.imshow("Model Input (28x28)", cv2.resize(thresh, (280, 280))
        )
59
60     if cv2.waitKey(1) & 0xFF == ord('q'):
61         break
62
63     except Exception as e:
64         print(f"图像获取或推理异常: {e}")
65         break
66
67     cv2.destroyAllWindows()

```

4.1.1 初版方案

首先需要完成把推理模型打包的过程，此后才能嵌入 ESP-S3。先说明模型来源。可以使用上一个方案直接获取的模型文件：

代码 4.3：下载 MNIST ONNX 模型

```

1 onnx_url = "https://github.com/onnx/models/raw/main/validated/vision/

```

```

        classification/mnist/model/mnist-12.onnx"
2 model_path = "mnist-12.onnx"
3 urllib.request.urlretrieve(onnx_url, model_path)

```

也可以自己搭建卷积神经网络训练一个模型文件。经过实际测试，这两个模型文件的效果没有太大区别：

代码 4.4：训练 MNIST 模型

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torchvision import datasets, transforms
5 from torch.utils.data import DataLoader
6
7 # MNIST CNN
8 class MNISTModel(nn.Module):
9     def __init__(self):
10         super(MNISTModel, self).__init__()
11         # 第一层卷积：输入 1 通道，输出 16 通道，卷积核 5x5
12         self.conv1 = nn.Conv2d(1, 16, kernel_size=5, stride=1, padding=2)
13         self.relu1 = nn.ReLU()
14         self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2) # 输出 14x14
15
16         # 第二层卷积：输入 16 通道，输出 32 通道，卷积核 5x5
17         self.conv2 = nn.Conv2d(16, 32, kernel_size=5, stride=1, padding=2)
18         self.relu2 = nn.ReLU()
19         self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2) # 输出 7x7
20
21         # 全连接分类层：32 * 7 * 7 = 1568 -> 10 个分类
22         self.fc1 = nn.Linear(32 * 7 * 7, 128)
23         self.relu3 = nn.ReLU()
24         self.fc2 = nn.Linear(128, 10)
25
26     def forward(self, x):
27         x = self.pool1(self.relu1(self.conv1(x)))
28         x = self.pool2(self.relu2(self.conv2(x)))
29         x = x.view(x.size(0), -1)
30         x = self.relu3(self.fc1(x))
31         x = self.fc2(x)
32         return x

```

```

33
34 # MNIST 数据集
35 transform = transforms.Compose([
36     transforms.ToTensor(),
37     transforms.Normalize((0.1307,), (0.3081,))
38 ])
39
40 train_dataset = datasets.MNIST(root='./data', train=True, download=True,
    transform=transform)
41 train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True
    )
42
43 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
44 model = MNISTModel().to(device)
45 criterion = nn.CrossEntropyLoss()
46 optimizer = optim.Adam(model.parameters(), lr=0.001)
47
48 model.train()
49 for epoch in range(3):
50     running_loss = 0.0
51     for i, (images, labels) in enumerate(train_loader):
52         images, labels = images.to(device), labels.to(device)
53
54         optimizer.zero_grad()
55         outputs = model(images)
56         loss = criterion(outputs, labels)
57         loss.backward()
58         optimizer.step()
59
60         running_loss += loss.item()
61     print(f"Epoch [{epoch+1}/3], Loss: {running_loss / len(train_loader):.4f
        }")
62
63 model.eval()
64 dummy_input = torch.randn(1, 1, 28, 28, device=device)
65
66 onnx_filename = "mnist-12.onnx"
67 torch.onnx.export(
68     model,
69     dummy_input,

```

```

70     onnx_filename,
71     export_params=True,
72     opset_version=12,
73     do_constant_folding=True,
74     input_names=['Input3'],
75     output_names=['Plus214_Output_0'],
76     dynamic_axes={
77         'Input3': {0: 'batch_size'},
78         'Plus214_Output_0': {0: 'batch_size'}
79     }
80 )
81
82 print(f"导出 ONNX 模型: {onnx_filename}")

```

接着通过 Edge Impulse 把模型打包成 Arduion IDE 可用的位流文件。具体来说：

1. 在 Edge Impulse 平台上传 mnist-12.onnx 模型文件。
2. 配置模型参数，包括输入输出维度、量化选项等。
3. 以 Arduion library 格式导出模型，生成位流文件。

即可获得库文件,在 Arduino IDE 中导入该库文件后,即可编写嵌入模型到 ESP32-S3 上运行的代码：

代码 4.5：嵌入模型到 ESP32-S3

```

1  #include <Test_inferencing.h>
2
3  #include "esp_camera.h"
4  #include "Arduino.h"
5
6  // 上一步导出库文件的头文件
7  #include <Test_inferencing.h>
8
9  // 参见方案雏形
10 #define PWDN_GPIO_NUM    -1
11 #define RESET_GPIO_NUM   -1
12 #define XCLK_GPIO_NUM    15
13 #define SIOD_GPIO_NUM     4
14 #define SIOC_GPIO_NUM    5
15

```

```

16 #define Y9_GPIO_NUM      16
17 #define Y8_GPIO_NUM      17
18 #define Y7_GPIO_NUM      18
19 #define Y6_GPIO_NUM      12
20 #define Y5_GPIO_NUM      10
21 #define Y4_GPIO_NUM       8
22 #define Y3_GPIO_NUM       9
23 #define Y2_GPIO_NUM      11
24
25 #define VSYNC_GPIO_NUM    6
26 #define HREF_GPIO_NUM     7
27 #define PCLK_GPIO_NUM    13
28
29 #define RAW_BUF_WIDTH    320
30 #define RAW_BUF_HEIGHT  240
31
32 static float features[EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT
    ];
33
34 static int raw_feature_get_data(size_t offset, size_t length, float *out_ptr
    ) {
35     memcpy(out_ptr, features + offset, length * sizeof(float));
36     return 0;
37 }
38
39 void setup() {
40     Serial.begin(115200);
41     delay(2000);
42     while (!Serial && millis() < 5000);
43
44     camera_config_t config;
45     config.ledc_channel = LEDC_CHANNEL_0;
46     config.ledc_timer = LEDC_TIMER_0;
47     config.pin_d0 = Y2_GPIO_NUM;
48     config.pin_d1 = Y3_GPIO_NUM;
49     config.pin_d2 = Y4_GPIO_NUM;
50     config.pin_d3 = Y5_GPIO_NUM;
51     config.pin_d4 = Y6_GPIO_NUM;
52     config.pin_d5 = Y7_GPIO_NUM;
53     config.pin_d6 = Y8_GPIO_NUM;

```

```

54     config.pin_d7 = Y9_GPIO_NUM;
55     config.pin_xclk = XCLK_GPIO_NUM;
56     config.pin_pclk = PCLK_GPIO_NUM;
57     config.pin_vsync = VSYNC_GPIO_NUM;
58     config.pin_href = HREF_GPIO_NUM;
59     config.pin_sccb_sda = SIOD_GPIO_NUM;
60     config.pin_sccb_scl = SIOC_GPIO_NUM;
61     config.pin_pwdn = PWDN_GPIO_NUM;
62     config.pin_reset = RESET_GPIO_NUM;
63     config.xclk_freq_hz = 100000000;
64     config.pixel_format = PIXFORMAT_GRAYSCALE;
65     config.frame_size = FRAMESIZE_QVGA;
66     config.jpeg_quality = 12;
67     config.fb_count = 1;
68     config.fb_location = CAMERA_FB_IN_PSRAM;
69
70     esp_err_t err = esp_camera_init(&config);
71     if (err != ESP_OK) {
72         Serial.printf("[Error] Camera initialized failed: 0x%x\n", err);
73         while (1);
74     }
75     sensor_t * s = esp_camera_sensor_get();
76
77     s->set_vflip(s, 1);
78
79     Serial.println("[Info] Model loaded\n");
80 }
81
82 void loop() {
83     camera_fb_t *fb = esp_camera_fb_get();
84     if (!fb) {
85         Serial.println("[Warn] Picture captured failed");
86         delay(500);
87         return;
88     }
89
90     int crop_size = 160;
91     int start_x = (RAW_BUF_WIDTH - crop_size) / 2;
92     int start_y = (RAW_BUF_HEIGHT - crop_size) / 2;
93     float scale = (float)crop_size / EI_CLASSIFIER_INPUT_WIDTH;

```

```

94
95     for (int y = 0; y < EI_CLASSIFIER_INPUT_HEIGHT; y++) {
96         for (int x = 0; x < EI_CLASSIFIER_INPUT_WIDTH; x++) {
97             int src_x = start_x + (int)(x * scale);
98             int src_y = start_y + (int)(y * scale);
99
100             uint8_t raw_pixel = fb->buf[src_y * RAW_BUF_WIDTH + src_x];
101
102             uint8_t val = (raw_pixel < 120) ? 255 : 0;
103
104             uint32_t rgb_pixel = (val << 16) | (val << 8) | val;
105
106             features[y * EI_CLASSIFIER_INPUT_WIDTH + x] = (float)rgb_pixel;
107         }
108     }
109
110     esp_camera_fb_return(fb);
111
112     signal_t signal;
113     signal.total_length = EI_CLASSIFIER_INPUT_WIDTH *
114         EI_CLASSIFIER_INPUT_HEIGHT;
115     signal.get_data = &raw_feature_get_data;
116
117     Serial.println("\n--- 28x28 input preview ---");
118     for (int y = 0; y < EI_CLASSIFIER_INPUT_HEIGHT; y++) {
119         for (int x = 0; x < EI_CLASSIFIER_INPUT_WIDTH; x++) {
120             float p = features[y * EI_CLASSIFIER_INPUT_WIDTH + x];
121             Serial.print(p > 0.0f ? "#" : ".");
122         }
123         Serial.println();
124     }
125
126     ei_impulse_result_t result = { 0 };
127     EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);
128
129     if (res != EI_IMPULSE_OK) {
130         Serial.printf("[Error] Local reasoning failed (%d)\n", res);
131         delay(1000);
132         return;
133     }

```

```

133
134     int max_idx = -1;
135     float max_val = -9999.0f;
136
137     for (size_t i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
138         if (result.classification[i].value > max_val) {
139             max_val = result.classification[i].value;
140             max_idx = i;
141         }
142     }
143
144     if (max_idx >= 0) {
145         int recognized_digit = max_idx;
146
147         Serial.printf("-> Recognized result: %d | Score: %.2f | time usage:
148             %d ms\n",
149             recognized_digit,
150             result.classification[max_idx].label,
151             max_val,
152             result.timing.classification);
153     }
154
155     delay(500);
156 }

```

此时，ESP32-S3-CAM 就可以在本地完成图像采集、预处理和 MNIST 模型推理，并通过串口输出识别结果。

4.2 终版方案

本方案与初版方案的区别主要在于模型选用的区别，在此简单说明本方案使用的 FOMO 模型与初版方案使用的 MNIST 模型的不同。

表 4.2: 模型比较

维度	MNIST	FOMO
任务类型	图像分类	目标检测
图像预处理要求	极高，涉及灰度化及硬二值化	较低，使用原始灰度像素点数据
ESP32 运行效率	较低，涉及很多模型外图像处理	较高，推理耗时固定

不同于初版方案，本方案模型的获取参考了开源项目 SSCMA[1]，只说明借助该项目获取模型文件的流程。

1. 获取模型训练仓库。
2. 访问 获取 COCO Mask 数据集并解压缩到项目根目录。
3. 执行

代码 4.6：训练 FOMO 模型

```
1 python tools/train.py \
2     configs/fomo/fomo_mobnetv2_0.35_x8_coco.py \
3     --cfg-options \
4     data_root=coco_mask/mask/ \
5     num_classes=2 \
6     train_ann=train/_annotations.coco.json \
7     val_ann=valid/_annotations.coco.json \
8     train_data=train/ \
9     val_data=valid/ \
10    epochs=50 \
11    height=192 \
12    width=192
```

4. 将模型文件导入 Edge Impulse 平台，进行必要的配置和打包。

由于模型功能不同，嵌入模型到 ESP32-S3 的代码也有差异，主要体现在图像预处理和结果处理部分：

代码 4.7：嵌入 FOMO 模型到 ESP32-S3

```
1 #include <Test_inferencing.h>
2
3 #include "esp_camera.h"
```

```

4 #include "Arduino.h"
5
6 #define PWDN_GPIO_NUM    -1
7 #define RESET_GPIO_NUM  -1
8 #define XCLK_GPIO_NUM    15
9 #define SIOD_GPIO_NUM    4
10 #define SIOC_GPIO_NUM    5
11
12 #define Y9_GPIO_NUM      16
13 #define Y8_GPIO_NUM      17
14 #define Y7_GPIO_NUM      18
15 #define Y6_GPIO_NUM      12
16 #define Y5_GPIO_NUM      10
17 #define Y4_GPIO_NUM      8
18 #define Y3_GPIO_NUM      9
19 #define Y2_GPIO_NUM      11
20
21 #define VSYNC_GPIO_NUM    6
22 #define HREF_GPIO_NUM     7
23 #define PCLK_GPIO_NUM    13
24
25 #define RAW_BUF_WIDTH    320
26 #define RAW_BUF_HEIGHT  240
27
28 static float features[EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT
29                       ];
30
31 static int raw_feature_get_data(size_t offset, size_t length, float *out_ptr
32 ) {
33     memcpy(out_ptr, features + offset, length * sizeof(float));
34     return 0;
35 }
36
37 void setup() {
38     Serial.begin(115200);
39     delay(2000);
40     while (!Serial && millis() < 5000);
41
42     camera_config_t config;
43     config.ledc_channel = LEDC_CHANNEL_0;

```

```

42     config.ledc_timer = LEDC_TIMER_0;
43     config.pin_d0 = Y2_GPIO_NUM;
44     config.pin_d1 = Y3_GPIO_NUM;
45     config.pin_d2 = Y4_GPIO_NUM;
46     config.pin_d3 = Y5_GPIO_NUM;
47     config.pin_d4 = Y6_GPIO_NUM;
48     config.pin_d5 = Y7_GPIO_NUM;
49     config.pin_d6 = Y8_GPIO_NUM;
50     config.pin_d7 = Y9_GPIO_NUM;
51     config.pin_xclk = XCLK_GPIO_NUM;
52     config.pin_pclk = PCLK_GPIO_NUM;
53     config.pin_vsync = VSYNC_GPIO_NUM;
54     config.pin_href = HREF_GPIO_NUM;
55     config.pin_sccb_sda = SIOD_GPIO_NUM;
56     config.pin_sccb_scl = SIOC_GPIO_NUM;
57     config.pin_pwdn = PWDN_GPIO_NUM;
58     config.pin_reset = RESET_GPIO_NUM;
59     config.xclk_freq_hz = 100000000;
60     config.pixel_format = PIXFORMAT_GRAYSCALE;
61     config.frame_size = FRAMESIZE_QVGA;
62     config.jpeg_quality = 12;
63     config.fb_count = 1;
64     config.fb_location = CAMERA_FB_IN_PSRAM;
65
66     esp_err_t err = esp_camera_init(&config);
67     if (err != ESP_OK) {
68         Serial.printf("[Error] Camera initialized failed: 0x%x\n", err);
69         while (1);
70     }
71     sensor_t * s = esp_camera_sensor_get();
72     s->set_vflip(s, 1);
73
74     Serial.println("[Info] FOMO Model and Camera initialized successfully\n"
75         );
76
77 void loop() {
78     camera_fb_t *fb = esp_camera_fb_get();
79     if (!fb) {
80         Serial.println("[Warn] Picture captured failed");

```

```

81     delay(500);
82     return;
83 }
84
85 int crop_size = 160;
86 int start_x = (RAW_BUF_WIDTH - crop_size) / 2;
87 int start_y = (RAW_BUF_HEIGHT - crop_size) / 2;
88 float scale = (float)crop_size / EI_CLASSIFIER_INPUT_WIDTH;
89
90 for (int y = 0; y < EI_CLASSIFIER_INPUT_HEIGHT; y++) {
91     for (int x = 0; x < EI_CLASSIFIER_INPUT_WIDTH; x++) {
92         int src_x = start_x + (int)(x * scale);
93         int src_y = start_y + (int)(y * scale);
94
95         uint8_t raw_pixel = fb->buf[src_y * RAW_BUF_WIDTH + src_x];
96
97         uint8_t val = raw_pixel;
98
99         uint32_t rgb_pixel = (val << 16) | (val << 8) | val;
100         features[y * EI_CLASSIFIER_INPUT_WIDTH + x] = (float)rgb_pixel;
101     }
102 }
103
104 esp_camera_fb_return(fb);
105
106 signal_t signal;
107 signal.total_length = EI_CLASSIFIER_INPUT_WIDTH *
108     EI_CLASSIFIER_INPUT_HEIGHT;
109 signal.get_data = &raw_feature_get_data;
110
111 ei_impulse_result_t result = { 0 };
112 EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);
113
114 if (res != EI_IMPULSE_OK) {
115     Serial.printf("[Error] Local inference failed (%d)\n", res);
116     delay(1000);
117     return;
118 }
119
120 bool bb_found = false;

```

```

120     Serial.printf("Inference timing: DSP %d ms, Classification %d ms\n",
121                   result.timing.dsp, result.timing.classification);
122
123 #if EI_CLASSIFIER_OBJECT_DETECTION == 1
124     for (size_t i = 0; i < EI_CLASSIFIER_OBJECT_DETECTION_COUNT; i++) {
125         auto bb = result.bounding_boxes[i];
126
127         if (bb.value < 0.5f) {
128             continue;
129         }
130
131         bb_found = true;
132         Serial.printf(" -> Detected target: %s (Score: %.2f) | Center Box [
133                       bb.label,
134                       bb.value,
135                       bb.x,
136                       bb.y,
137                       bb.width,
138                       bb.height);
139     }
140 #endif
141
142     if (!bb_found) {
143         Serial.println(" -> No target detected.");
144     }
145
146     delay(200);
147 }

```

此时，ESP32-S3-CAM 就可以在本地完成图像采集、预处理和 FOMO 模型推理，并通过串口输出识别结果。

5 运行效果

由于方案雏形不涉及边缘 AI 计算且大部分工作都是已有代码的增量工作，因此不再赘述。初版方案和终版方案的运行效果如下。

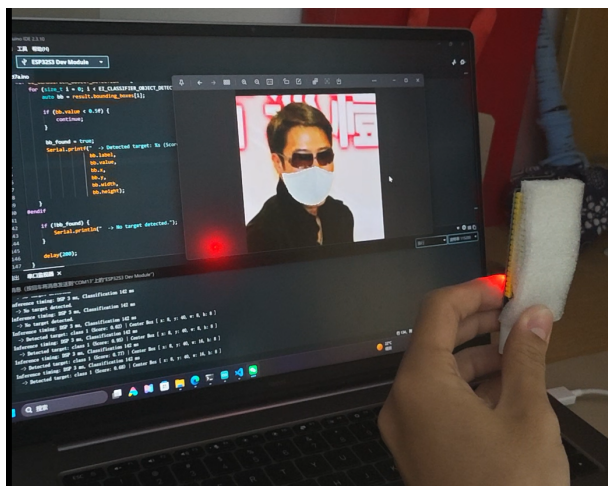
5.1 初版方案



[点击此处查看演示视频](#)



5.2 终版方案



[点击此处查看演示视频](#)



参考文献

- [1] SSCMA. 专注于嵌入式人工智能的开源项目. SSCMA, 2024. 访问日期: 2026-10-06.
- [2] Espressif Systems. *ESP 硬件设计指南*. Espressif Systems, 2024. 在线文档, 访问日期: 2026-10-05.
- [3] 宇凡微. 单片机和 *soc* 的区别. 宇凡微电子, 2023. 知乎专栏, 访问日期: 2026-10-05.